

Competitive Programming

Section 39 Editorial: Heaps and Priority Queues

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Running Median	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. Course Rooms	6
3.1	Restatement	6
3.2	Algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	7
4	C. K Smallest Pair Sums	8
4.1	Restatement	8
4.2	Algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	9
5	Testing discussion	10

1 Editorial overview

Problem	Main idea
A. Running Median	See full algorithm and code below.
B. Course Rooms	See full algorithm and code below.
C. K Smallest Pair Sums	See full algorithm and code below.

2 A. Running Median

2.1 Restatement

After every inserted number, print the lower median of the prefix.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. \square

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
```

```

if (!(cin >> n)) return 0;
priority_queue<long long> low;
priority_queue<long long, vector<long long>, greater<long long>> high;
for (int i = 0; i < n; i++) {
    long long x;
    cin >> x;
    if (low.empty() || x <= low.top()) low.push(x);
    else high.push(x);
    if (low.size() < high.size()) {
        low.push(high.top());
        high.pop();
    }
    if (low.size() > high.size() + 1) {
        high.push(low.top());
        low.pop();
    }
    if (i) cout << ' ';
    cout << low.top();
}
cout << '\n';
return 0;
}

```

2.6 Full Python solution

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    low, high = [], []
    out = []
    for x in data[1:1 + n]:
        if not low or x <= -low[0]:
            heapq.heappush(low, -x)
        else:
            heapq.heappush(high, x)
        if len(low) < len(high):
            heapq.heappush(low, -heapq.heappop(high))
        if len(low) > len(high) + 1:
            heapq.heappush(high, -heapq.heappop(low))
        out.append(str(-low[0]))
    print(" ".join(out))

if __name__ == "__main__":
    main()

```

3 B. Course Rooms

3.1 Restatement

Given half-open intervals $[l, r)$, print the minimum number of rooms needed.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
```

```

if (!(cin >> n)) return 0;
vector<pair<int,int>> intervals(n);
for (auto &[l, r] : intervals) cin >> l >> r;
sort(intervals.begin(), intervals.end());
priority_queue<int, vector<int>, greater<int>> pq;
int ans = 0;
for (auto [l, r] : intervals) {
    while (!pq.empty() && pq.top() <= l) pq.pop();
    pq.push(r);
    ans = max(ans, (int)pq.size());
}
cout << ans << '\n';
return 0;
}

```

3.6 Full Python solution

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    intervals = [(data[i], data[i + 1]) for i in range(1, 2 * n + 1, 2)]
    intervals.sort()
    heap = []
    ans = 0
    for l, r in intervals:
        while heap and heap[0] <= l:
            heapq.heappop(heap)
        heapq.heappush(heap, r)
        ans = max(ans, len(heap))
    print(ans)

if __name__ == "__main__":
    main()

```

4 C. K Smallest Pair Sums

4.1 Restatement

Two arrays are sorted. Print the k smallest values of $a[i]+b[j]$.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
```

```

if (!(cin >> n >> m >> k)) return 0;
vector<long long> a(n), b(m);
for (auto &x : a) cin >> x;
for (auto &x : b) cin >> x;
using State = tuple<long long,int,int>;
priority_queue<State, vector<State>, greater<State>> pq;
for (int i = 0; i < n; i++) pq.emplace(a[i] + b[0], i, 0);
for (int t = 0; t < k; t++) {
    auto [s, i, j] = pq.top();
    pq.pop();
    if (t) cout << ' ';
    cout << s;
    if (j + 1 < m) pq.emplace(a[i] + b[j + 1], i, j + 1);
}
cout << '\n';
return 0;
}

```

4.6 Full Python solution

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, k = data[0], data[1], data[2]
    a = data[3:3 + n]
    b = data[3 + n:3 + n + m]
    heap = [(a[i] + b[0], i, 0) for i in range(n)]
    heapq.heapify(heap)
    out = []
    for _ in range(k):
        s, i, j = heapq.heappop(heap)
        out.append(str(s))
        if j + 1 < m:
            heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
    print(" ".join(out))

if __name__ == "__main__":
    main()

```

5 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.